

12주차 예습과제

🕒 과제 종류	예습과제
≡ 범위	자연어처리와 컴퓨터비전 심층학습 7장 (~GPT)

- [참고] 실습 과정에서 버전 충돌로 인해 각종 오류가 많았지만 아래 버전으로 실습 완료했습니다!

실행 환경 정보

- 실행 환경: Colab
- Python: 3.12.12
- torchtext: 0.18.0
- PyTorch: 2.3.0 (torchtext 0.18은 2.3.0과 호환)
- torchdata: 0.9.0

Chapter7. 트랜스포머

1. 트랜스포머(Transformer)

- Attention is All You Need (구글 브레인, 2017) 을 통해 소개된 신경망 아키텍처
- 기존 순환 신경망(RNN)과 달리 **병렬**로 입력 시퀀스를 처리.
- 순차 처리나 반복 연결 없이 **셀프 어텐션(Self-Attention)**을 기반으로 입력 토큰 간의 관계를 직접 모델링.
- 장기적인 종속성을 포함하는 기계 번역, 텍스트 요약 등에 매우 효과적

모델 학습 구조 및 방향성

모델	구조	학습 방법	방향성
BERT	인코더	오토 인코딩 (빈칸 채우기)	양방향
GPT	디코더	자기 회귀 (다음 단어 예측)	단방향
BART	인코더+디코더	오토 인코딩 + 자기 회귀	양방향 + 단방향
ELECTRA	인코더 판별기	대체 토큰 탐지	양방향
T5	인코더+디코더	다양한 자연어 처리 작업 학습	양방향

2. 트랜스포머 모델 상세 구조

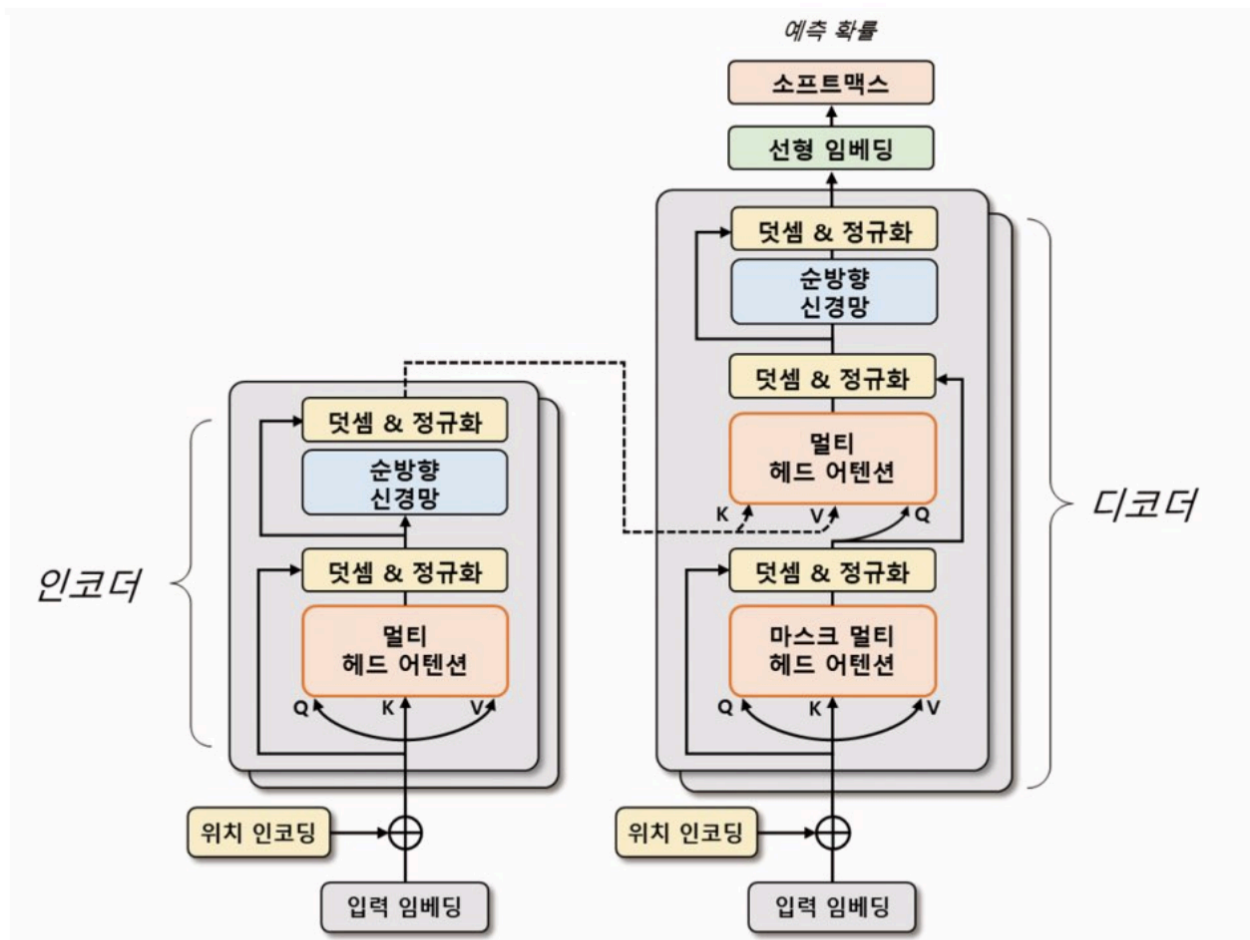


그림 7.2 트랜스포머 모델 구조(Q: 쿼리, K: 키, V: 값)

- 트랜스포머는 **인코더와 디코더**로 구성되며, 각각 N개의 블록으로 이루어짐.

- **멀티 헤드 어텐션 (Multi-Head Attention)**

- 입력 시퀀스에서 **쿼리(Query)**, **키(Key)**, **값(Value)** 벡터를 정의하여 **셀프 어텐션 (Self-Attention)**을 수행함.
- 작동 원리: 쿼리와 키의 유사도를 계산한 후, 이를 가중치로 사용하여 값 벡터를 합산함.
- 효과: 단어 간 상호작용을 파악하여 임베딩 벡터를 갱신함.

- **순방향 신경망 (Feed-Forward Network)**

- 어텐션을 거친 임베딩 벡터를 더욱 고도화함.
- 여러 선형 계층으로 구성되며, **가중치 곱, 편향 덧셈, 활성화 함수**를 적용함.
- 입력 시퀀스 각 단어의 의미를 명확히 파악하도록 학습됨.

- 트랜스포머는 입력 데이터를 **소스(Source)**와 **타겟(Target)**으로 나누어 처리

- **소스(Source)**: 참조하는 언어 (예: 번역할 영어 문장)
- **타겟(Target)**: 생성하는 언어 (예: 번역된 한글 문장)

- **인코더 (Encoder)**

- 소스 시퀀스 데이터에 **위치 인코딩(Positional Encoding)**을 더해 입력받음.
- 입력 데이터 간의 관계를 함축한 **출력 벡터**를 생성하여 디코더로 전달함.

- **디코더 (Decoder)**

- **마스크 멀티 헤드 어텐션(Masked Multi-Head Attention)**을 사용하여 타겟 데이터를 순차적으로 생성함.
- 데이터 간의 관계를 더 정교하게 파악하기 위해 **인코더가 생성한 출력 벡터 정보를 참조**함.
- 최종 출력은 선형 임베딩을 통해 변환되어 챗봇, 번역, 음성 인식 등에 활용됨.

2.1 주요 구성 요소

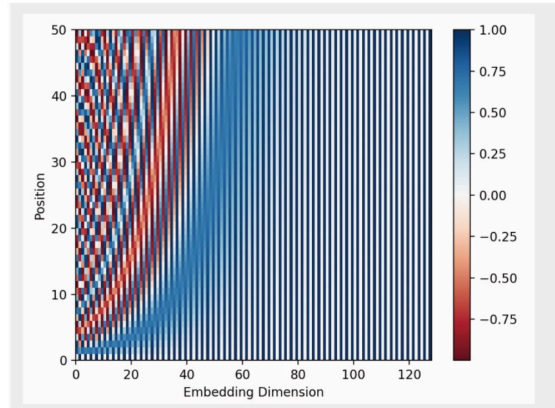
1. 입력 임베딩 & 위치 인코딩 (Positional Encoding):

- 트랜스포머는 순환 신경망(RNN)과 달리 입력 시퀀스를 병렬로 처리하므로, 기본적으로 단어의 순서 정보를 가지고 있지 않음.
- 따라서 단어의 순서 정보를 모델에 반영하기 위해 임베딩 벡터에 위치 정보를 추가해야 하며, 이를 위해 **위치 인코딩(Positional Encoding)** 방식을 사용함.
- 입력 시퀀스의 순서 정보를 전달하기 위해 각 단어의 위치를 나타내는 벡터를 생성하고, 이를 기존 임베딩 벡터에 더해줌
- 위치 인코딩 벡터는 **sin 함수와 cos 함수**를 사용하여 생성됨.

⇒ 각 토큰의 위치를 각도로 표현하여 계산하므로, 토큰의 위치마다 서로 다른 임베딩 벡터가 생성됨.

결과적으로 모델이 임베딩 벡터와 결합된 위치 정보를 통해 단어의 순서를 학습할 수 있게 된다.

출력 결과



2. 특수 토큰

- 트랜스포머는 단어 토큰 이외의 특수 토큰을 활용해 문장을 표현함.
- 특수 토큰 : 입력 시퀀스의 시작과 끝 / 마스킹 영역으로 사용.

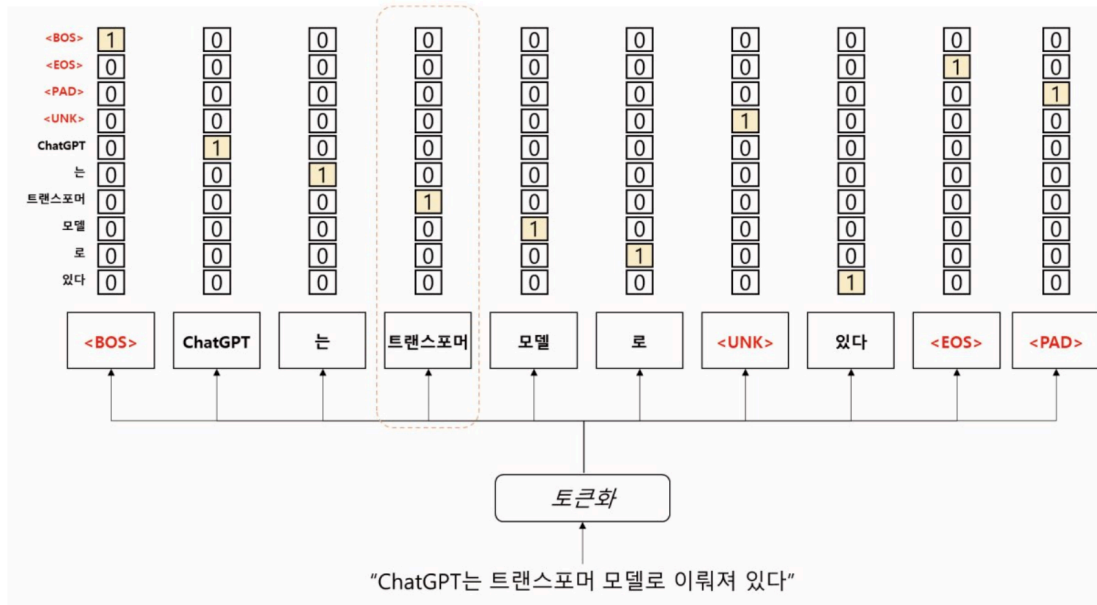


그림 7.3 특수 토큰을 활용한 문장 토큰 배열

- **<BOS>**: 문장의 시작 (Beginning of Sentence).
- **<EOS>**: 문장의 끝 (End of Sentence).
- **<UNK>**: 모르는 단어 (Unknown).
- **<PAD>**: 문장 길이를 맞추기 위한 패딩 (Padding).

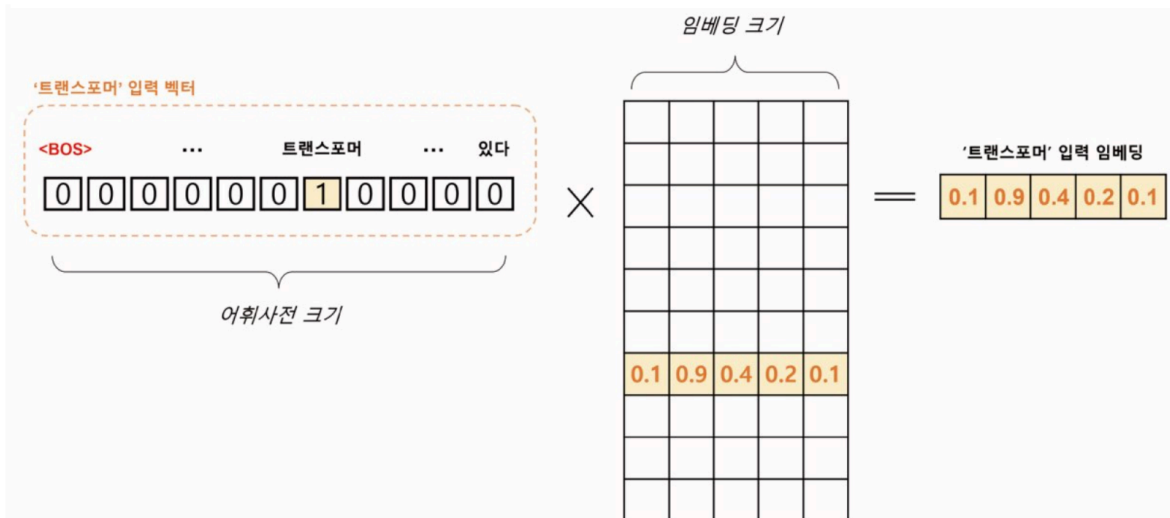


그림 7.4 입력 임베딩 산출 과정

임베딩 변환 원리 (Word2Vec과 동일)

- 어휘 사전의 크기를 V , 입력 임베딩 차원을 d 로 정의함.
- 특정 단어(예: '트랜스포머')의 원-핫 벡터 크기는 $[1, V]$ 임.
- 이 원-핫 벡터는 임베딩 행렬 $[V, d]$ 와 연산되어 $[1, d]$ 크기의 벡터로 변환됨.
- N 개의 문장이 최대 S 개의 토큰 길이를 가진다고 가정할 때, $[N, S, V]$ 크기의 원-핫 벡터 텐서는 **$[N, S, d]$ 크기의 임베딩 텐서**로 변환됨.
- 변환된 임베딩 텐서는 모델의 입력으로 사용됨.
- 이 텐서는 트랜스포머의 모든 계층에서 공유됨.

3. 멀티 헤드 어텐션 (Multi-Head Attention):

- 입력 시퀀스에서 **쿼리(Query)**, **키(Key)**, **값(Value)** 벡터를 정의하여 관계를 계산

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

- $\sqrt{d_k}$ 로 나누는 이유는 차원이 커질 때 스코어 값이 커져 기울기 소실이 발생하는 것을 방지하기 위함.

4. 순방향 신경망 (Feed-Forward Networks):

- 선형 계층과 활성화 함수(ReLU 등)로 구성되어 임베딩 벡터를 고도화

5. 덧셈 & 정규화 (Add & Norm):\

- 잔차 연결(Residual Connection)을 통해 기울기 소실 완화 및 계층 정규화 적용

2.2 인코더 (Encoder)

- 입력 시퀀스의 정보를 추출하여 디코더로 전달.
- **셀프 어텐션**: 입력된 문장 내의 단어 간 관계를 파악.

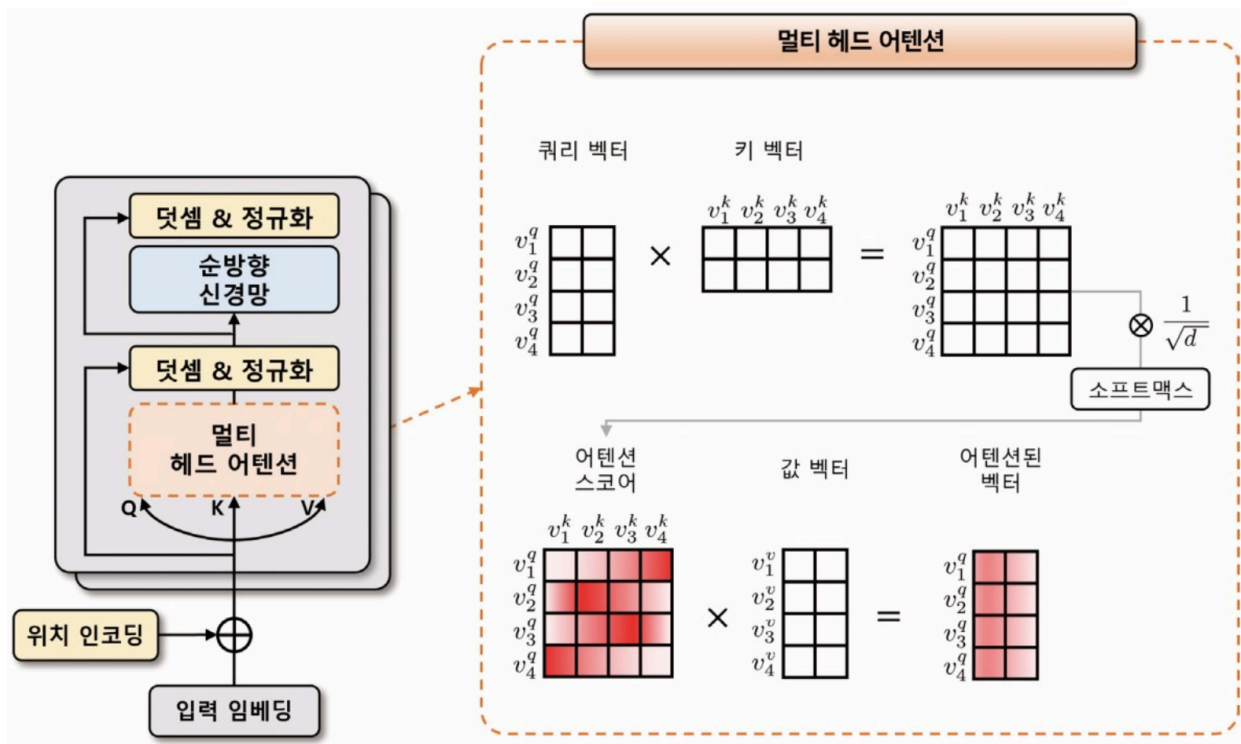


그림 7.5 트랜스포머 인코더 블록 연산 과정

- 입력: 위치 인코딩된 임베딩
- 멀티헤드 어텐션
 - 입력 벡터를 세 개로 분리 → Query(Q), Key(K), Value(V)
 - Q·K 유사도를 Softmax로 계산, Value에 가중합
 - 여러 개의 헤드(k개) 병렬 수행 → 정보 다양성 확보
 - 결과 $[N, S, d]$ 유지
- Feed Forward Network
 - 선형변환 + ReLU → 덧셈 & 정규화 (Residual + LayerNorm)
- 출력은 디코더의 입력으로 전달 (Key, Value로 사용)

2.3 디코더 (Decoder)

- 'ChatGPT는 트랜스포머 모델로 이뤄져 있다.' 문장에 대한 디코더 입력과 출력

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD]

- 출력 단계
 - [IN, S, d] → 선형 변환 + Softmax → 각 단어별 확률 예측
 - PAD/빈칸 채워 문장 순차적 생성

4. GPT (Generative Pre-trained Transformer)

- OpenAI에서 개발한 트랜스포머 디코더(Decoder) 기반의 언어 모델

4.1 GPT-1 (2018)

- 구조: 트랜스포머 디코더 12층 사용. 인코더-디코더 어텐션 제거.
- 학습: BookCorpus 데이터로 비지도 사전 학습 후 다운스트림 작업(분류, 추론 등)에 미세 조정(Fine-tuning) 수행.
- 특수 토큰: 작업에 따라 <Start>, <Extract>, <Delim> 등을 사용하여 입출력 구조 통일.

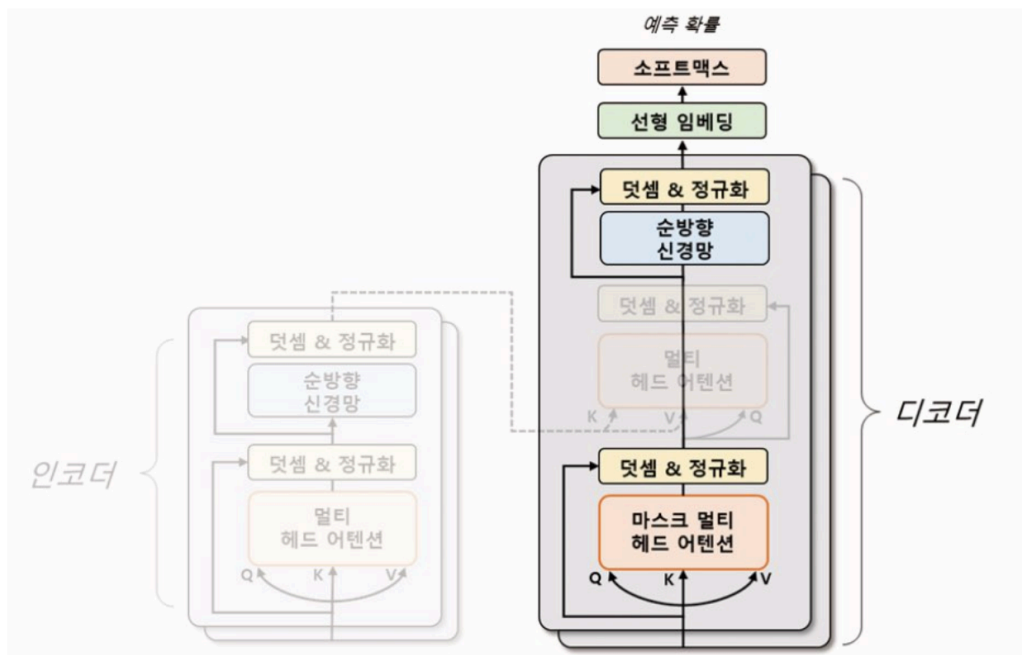


그림 7.7 트랜스포머와 GPT-1 모델 비교

4.2 GPT-2 (2019)

- **규모:** 파라미터 15억 개(GPT-1의 10배 이상), 레이어 48개, 입력 길이 1,024토큰.
- **데이터:** 40GB의 웹 텍스트 데이터 사용.
- **특징:** 제로-샷(Zero-shot) 학습 가능. 별도의 미세 조정 없이 프롬프트만으로 작업 수행.

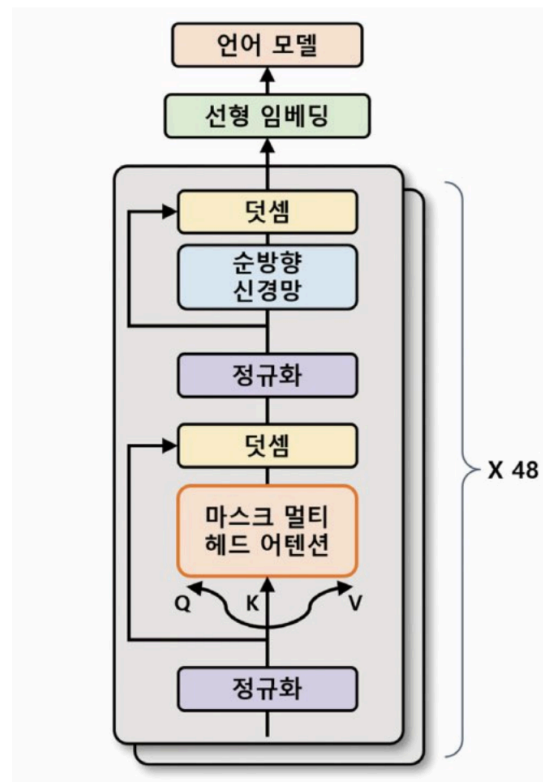


그림 7.10 GPT-2의 모델 구조

4.3 GPT-3 (2020)

- **규모:** 파라미터 1,750억 개, 레이어 96개, 입력 길이 2,048토큰.
- **데이터:** 45TB의 방대한 데이터로 학습. 희소 어텐션(Sparse Attention) 도입.
- **프롬프트 학습:** 퓨-샷(Few-shot) 등을 통해 별도 학습 없이 자연어 처리 작업 수행.
- **한계:** 높은 비용, 환각(Hallucination) 현상 및 편향성 존재.

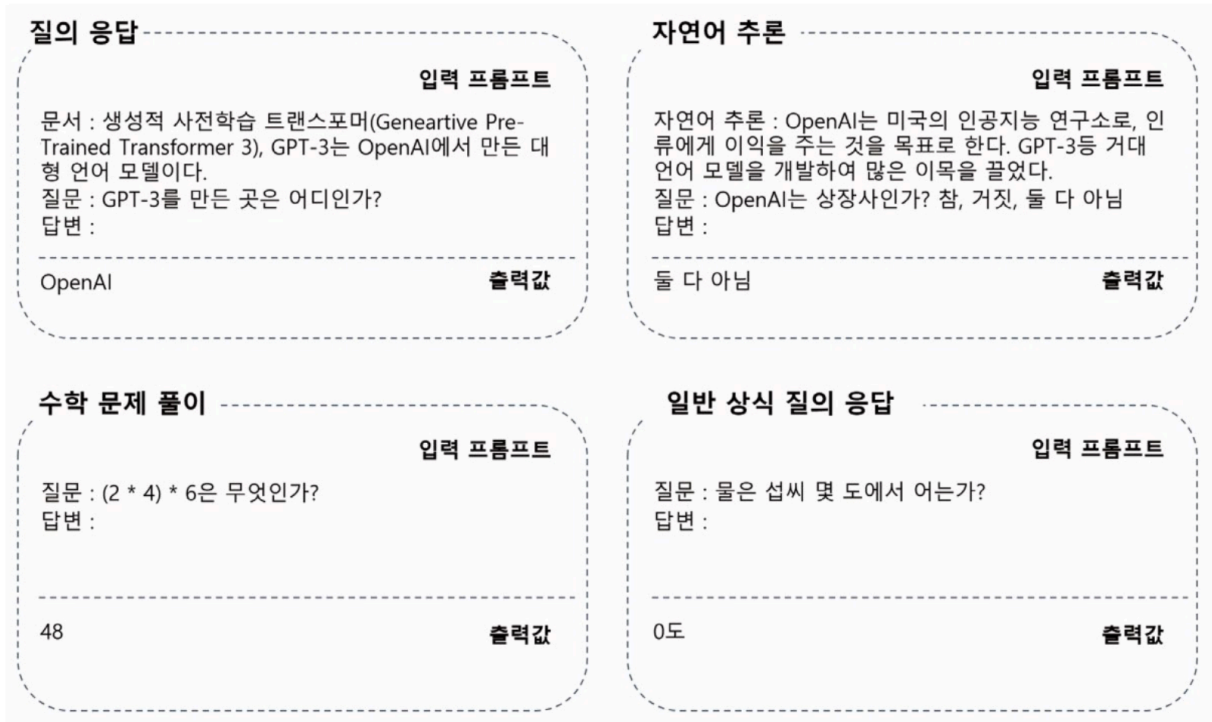


그림 7.11 GPT-3의 다운스트림 작업 프롬프트 예시

4.4 GPT-3.5 (InstructGPT) & RLHF

- **RLHF (Reinforcement Learning from Human Feedback):** 인간의 피드백을 활용한 강화 학습 도입.
- **학습 3단계:**
 1. **SFT (Supervised Fine-Tuning):** 사람이 작성한 모범 답안으로 모델 미세 조정.
 2. **보상 모델 학습:** 모델이 생성한 여러 답변에 사람이 랭킹을 매겨 보상 모델 학습.
 3. **PPO 강화 학습:** 보상 모델의 점수를 최대화하도록 언어 모델 최적화.
- **결과:** 더 자연스럽게 지시를 잘 따르는 모델(ChatGPT의 기반) 개발.

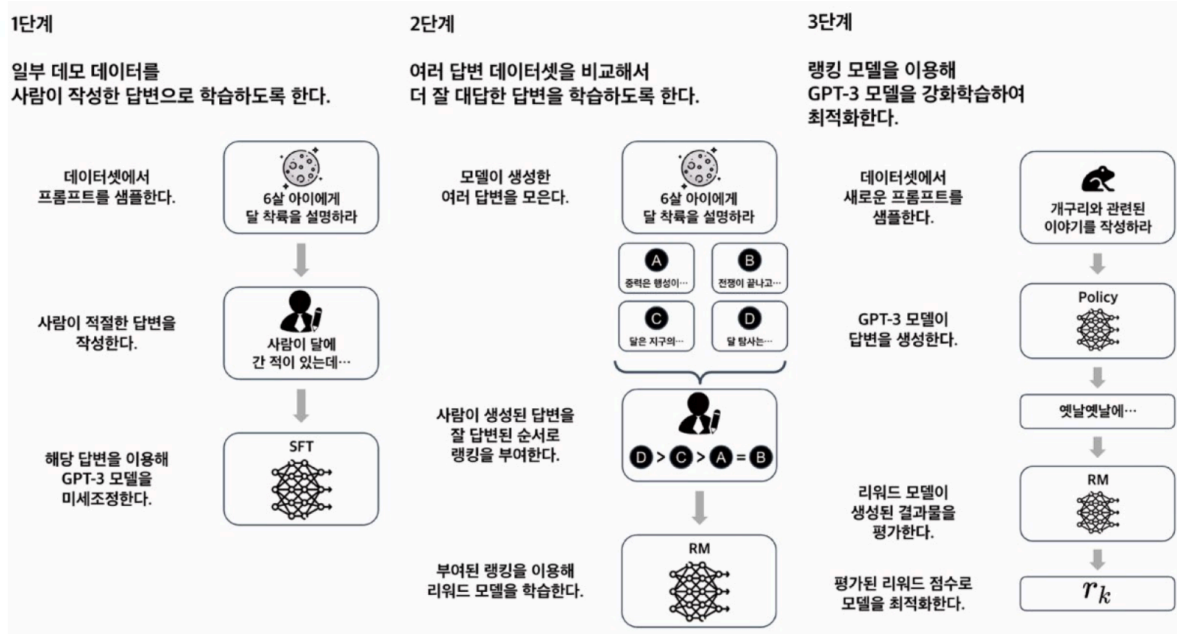


그림 7.12 RLHF GPT-3 학습 방법

4.5 GPT-4

- **멀티모달:** 텍스트뿐만 아니라 **이미지**도 입력받아 처리 가능.
- **성능:** 입력 토큰 32,768개로 확장(약 25,000단어). 변호사 시험 상위 10% 등 전문 분야에서 성능 대폭 향상.
- **환각 개선:** 이전 모델 대비 사실성 및 정확도 향상.